

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №7
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Обход графа в глубину»

Выполнил:

студенты группы 24ВВВ2

Бауэр А.А.

Гусев Е.В.

Медведев Е.А.

Принял:

к.т.н. доцент Юрова О.В.

к.т.н. Деев М.В.

Пенза 2025

Название

Обход графа в глубину.

Цель работы

Научиться производить обход в графа в глубину.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.
3. *Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2

1. Для матричной формы представления графов выполните операцию:
 - а) отождествления вершин
 - б) стягивания ребра
 - в) расщепления вершины
 Номера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнения операции выведите на экран.
2. *Для матричной формы представления графов выполните преобразование рекурсивной реализации обхода графа к не рекурсивной

Описание метода решения задачи

Задание 1

Метод решения основан на алгоритме обхода графа в глубину с использованием двух структур данных: матрицы смежности и списков смежности. Для матричного представления строится симметричная матрица $n \times n$ с нулевой диагональю, где элементы указывают наличие связей между вершинами. Алгоритм рекурсивно посещает вершины, перемещаясь к первым непосещенным смежным вершинам с использованием массива пометок.

При списковом представлении для каждой вершины формируется связный список смежных вершин с симметричным добавлением ребер. Обход выполняется рекурсивно через обход списков смежности. Оба подхода обеспечивают полный обход всех компонент связности через внешний цикл по непосещенным вершинам и демонстрируют идентичность результатов при различном внутреннем представлении графа.

Задание 2

Метод решения основан на нерекурсивной реализации обхода графа в глубину с использованием стека. Граф представляется в виде матрицы

смежности размером $n \times n$, где симметричные элементы указывают на наличие связей в неориентированном графе без петель.

Алгоритм начинает обход с заданной вершины, помещая ее в стек. На каждом шаге извлекается вершина из стека, отмечается как посещенная и в обратном порядке добавляются в стек все ее непосещенные смежные вершины. Это обеспечивает тот же порядок обхода, что и рекурсивная версия. Использование стека позволяет избежать рекурсивных вызовов и связанных с ними ограничений.

Листинг

Файл LR7_11_12.cpp (1 задание 1-2)

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#define MAX_VERTICES 20// Максимальное количество вершин в графе

int visited[MAX_VERTICES];// Массив для отслеживания посещенных
вершин

//Рекурсивная функция обхода в глубину (Depth-First Search)
void DFS(int graph[MAX_VERTICES][MAX_VERTICES], int vertex, int
num_vertices) {
    visited[vertex] = 1;// Помечаем текущую вершину как посещенную
    printf("%d ", vertex + 1);// Выводим номер вершины (начиная с 1)
    //Просматриваем все смежные вершины
    for (int i = 0; i < num_vertices; i++) {
        //Если есть связь с вершиной i и она еще не посещена
        if (graph[vertex][i] == 1 && !visited[i]) {
            DFS(graph, i, num_vertices);// Рекурсивно переходим к
вершине i
        }
    }
}

//Функция для генерации случайной матрицы смежности
void generateRandomGraph(int graph[MAX_VERTICES][MAX_VERTICES], int
num_vertices) {
    srand(time(NULL));
    //Заполняем матрицу смежности
    for (int i = 0; i < num_vertices; i++) {
        for (int j = i; j < num_vertices; j++) {
            if (i == j) {
                graph[i][j] = 0;// Нет петель
            }
            else {
                //Случайным образом определяем наличие связи между
вершинами
                graph[i][j] = rand() % 2;
                graph[j][i] = graph[i][j];// Граф неориентированный
            }
        }
    }
}
```

```

    }
}

//Функция для вывода матрицы смежности на экран
void printGraph(int graph[MAX_VERTICES][MAX_VERTICES], int
num_vertices) {
    printf("\nМатрица смежности:\n");
    printf("    ");
    for (int i = 0; i < num_vertices; i++) {
        printf("%2d ", i + 1); // Нумерация столбцов с 1
    }
    printf("\n");
    //Вывод самой матрицы
    for (int i = 0; i < num_vertices; i++) {
        printf("%2d ", i + 1);
        for (int j = 0; j < num_vertices; j++) {
            printf("%2d ", graph[i][j]);
        }
        printf("\n");
    }
}

int main() {
    setlocale(LC_ALL, "ru");
    int num_vertices; // Количество вершин в графе
    int graph[MAX_VERTICES][MAX_VERTICES]; // Матрица смежности графа
    printf("Введите количество вершин графа (от 1 до %d): ",
MAX_VERTICES);
    scanf("%d", &num_vertices);
    if (num_vertices < 1 || num_vertices > MAX_VERTICES) {
        printf("Ошибка: количество вершин должно быть от 1 до %d\n",
MAX_VERTICES);
        return 1;
    }
    //Инициализация массива посещенных вершин
    for (int i = 0; i < num_vertices; i++) {
        visited[i] = 0; // 0 - вершина не посещена, 1 - посещена
    }

    //Генерация случайного графа
    generateRandomGraph(graph, num_vertices);

    //Вывод матрицы смежности
    printGraph(graph, num_vertices);

    //Выполнение обхода в глубину
    printf("\nПорядок обхода в глубину: ");
    for (int i = 0; i < num_vertices; i++) {
        //Если вершина еще не посещена, начинаем обход из нее
        if (!visited[i]) {
            DFS(graph, i, num_vertices);
        }
    }
    printf("\n");
    return 0;
}

```

Файл 7LR_13.cpp (1 задание 3)

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#define MAX_VERTICES 20

//Структура для узла списка смежности
typedef struct Node {
    int vertex; // Номер вершины, с которой есть связь
    struct Node* next; // Указатель на следующий узел
} Node;

Node* adjacencyList[MAX_VERTICES]; // Массив указателей на списки
смежности для каждой вершины
int visited[MAX_VERTICES]; // Массив для отслеживания посещенных
вершин

//Функция добавления ребра в список смежности
void addEdge(int src, int dest) {
    //Создаем новый узел для направления src -> dest
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->vertex = dest;
    newNode->next = adjacencyList[src];
    adjacencyList[src] = newNode;
    //Для неориентированного графа добавляем обратное ребро dest ->
src
    newNode = (Node*)malloc(sizeof(Node));
    newNode->vertex = src;
    newNode->next = adjacencyList[dest];
    adjacencyList[dest] = newNode;
}

//Функция генерации графа
void generateRandomGraphList(int num_vertices) {
    srand(time(NULL));
    //Инициализация всех списков смежности как пустых
    for (int i = 0; i < num_vertices; i++) {
        adjacencyList[i] = NULL;
    }
    //Генерация случайных ребер между вершинами
    for (int i = 0; i < num_vertices; i++) {
        for (int j = i + 1; j < num_vertices; j++) {
            if (rand() % 2 == 1) {
                addEdge(i, j);
            }
        }
    }
}

//Рекурсивная функция обхода в глубину для списков смежности
void DFS_list(int vertex) {
    visited[vertex] = 1; // Помечаем текущую вершину как посещенную
    printf("%d ", vertex + 1); // Выводим номер вершины (начиная с 1)
    //Проходим по всем смежным вершинам в списке смежности

```

```

Node* temp = adjacencyList[vertex];
while (temp != NULL) {
    int adjVertex = temp->vertex; // Получаем номер смежной
вершины
    //Если смежная вершина еще не посещена, рекурсивно переходим
к ней
    if (!visited[adjVertex]) {
        DFS_list(adjVertex);
    }
    temp = temp->next; // Переходим к следующему элементу списка
}

//Функция вывода списков смежности
void printAdjacencyList(int num_vertices) {
    printf("Списки смежности:\n");
    for (int i = 0; i < num_vertices; i++) {
        printf("Вершина %d: ", i + 1);
        Node* temp = adjacencyList[i];
        while (temp != NULL) {
            printf("%d ", temp->vertex + 1);
            temp = temp->next;
        }
        printf("\n");
    }
}

int main() {
    setlocale(LC_ALL, "ru");
    int num_vertices; // Количество вершин в графе
    printf("Введите количество вершин графа (от 1 до %d): ",
MAX_VERTICES);
    scanf("%d", &num_vertices);
    if (num_vertices < 1 || num_vertices > MAX_VERTICES) {
        printf("Ошибка: количество вершин должно быть от 1 до %d\n",
MAX_VERTICES);
        return 1;
    }
    // Инициализация массива посещенных вершин
    for (int i = 0; i < num_vertices; i++) {
        visited[i] = 0; // 0 - вершина не посещена, 1 - посещена
    }
    generateRandomGraphList(num_vertices);
    printAdjacencyList(num_vertices);
    // Выполнение обхода в глубину
    printf("Порядок обхода в глубину: ");
    for (int i = 0; i < num_vertices; i++) {
        // Если вершина еще не посещена, начинаем обход из нее
        if (!visited[i]) {
            DFS_list(i);
        }
    }
    printf("\n");
    return 0;
}

```

Файл 7Lr_2_star.cpp (2 задание)

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#define MAX_VERTICES 20// Количество вершин
#define STACK_SIZE 100

int stack[STACK_SIZE];// Стек для хранения вершин при обходе
int top = -1;// Вершина стека

//Функция добавления вершины в стек
void push(int vertex) {
    if (top < STACK_SIZE - 1) {
        stack[++top] = vertex;
    }
}

//Функция извлечения вершины из стека
int pop() {
    if (top >= 0) {
        return stack[top--];
    }
    return -1; // Возвращаем -1 если стек пуст
}

//Функция проверки пустоты стека
int isEmpty() {
    return top == -1;
}

//Нерекурсивный обход в глубину
void DFS_nonRecursive(int graph[MAX_VERTICES][MAX_VERTICES], int
start_vertex, int num_vertices) {
    int visited[MAX_VERTICES];// Массив для отслеживания посещенных
вершин
    //Инициализация массива посещенных вершин
    for (int i = 0; i < num_vertices; i++) {
        visited[i] = 0;
    }
    //Помещаем начальную вершину в стек и отмечаем как посещенную
    push(start_vertex);
    visited[start_vertex] = 1;
    printf("Порядок обхода в глубину (нерекурсивный): ");
    //Пока стек не пуст, продолжаем обход
    while (!isEmpty()) {
        int current_vertex = pop();// Извлекаем вершину из стека
        printf("%d ", current_vertex + 1);// Выводим номер вершины
(начиная с 1)
        //Обходим смежные вершины в обратном порядке (для сохранения
порядка как в рекурсивной версии)
        for (int i = num_vertices - 1; i >= 0; i--) {
            // Если есть связь и вершина не посещена
            if (graph[current_vertex][i] == 1 && !visited[i]) {
                push(i);// Добавляем вершину в стек
            }
        }
    }
}

```

```

        visited[i] = 1; // Отмечаем как посещенную
    }
}
}
printf("\n");
}

//Функция для генерации случайной матрицы смежности
void generateRandomGraph(int graph[MAX_VERTICES][MAX_VERTICES], int
num_vertices) {
    srand(time(NULL));
    //Заполнение матрицы смежности
    for (int i = 0; i < num_vertices; i++) {
        for (int j = i; j < num_vertices; j++) {
            if (i == j) {
                graph[i][j] = 0; // Нет петель
            }
            else {
                graph[i][j] = rand() % 2;
                graph[j][i] = graph[i][j];
            }
        }
    }
}

//Функция для вывода матрицы смежности
void printGraph(int graph[MAX_VERTICES][MAX_VERTICES], int
num_vertices) {
    printf("\nМатрица смежности:\n");
    printf("    ");
    for (int i = 0; i < num_vertices; i++) {
        printf("%2d ", i + 1);
    }
    printf("\n");
    for (int i = 0; i < num_vertices; i++) {
        printf("%2d ", i + 1);
        for (int j = 0; j < num_vertices; j++) {
            printf("%2d ", graph[i][j]);
        }
        printf("\n");
    }
}

int main() {
    setlocale(LC_ALL, "ru");
    int num_vertices; // Количество вершин в графе
    int graph[MAX_VERTICES][MAX_VERTICES]; // Матрица смежности
    printf("Введите количество вершин графа (от 1 до %d): ",
MAX_VERTICES);
    scanf("%d", &num_vertices);
    if (num_vertices < 1 || num_vertices > MAX_VERTICES) {
        printf("Ошибка: количество вершин должно быть от 1 до %d\n",
MAX_VERTICES);
        return 1;
    }
    generateRandomGraph(graph, num_vertices);
}

```



```

printGraph(graph, num_vertices);
//Выполнение нерекурсивного обхода в глубину
DFS_nonRecursive(graph, 0, num_vertices); // Начинаем обход с
вершины 0
return 0;
}

```

Текст к программе

1. LR7_11_12.cpp (1 задание 1-2)

Данная программа реализует рекурсивный обход графа в глубину. Граф представляется в виде матрицы смежности, которая генерируется случайным образом. Алгоритм начинает обход из каждой непосещенной вершины, рекурсивно переходя к смежным вершинам и выводя порядок их посещения. Программа обеспечивает полный обход всех компонент связности графа.

2. 7LR_13_star.cpp (1 задание 3)

Программа реализует рекурсивный обход графа в глубину с использованием списков смежности. Граф генерируется случайным образом, каждое ребро симметрично добавляется в списки смежности обеих вершин. Алгоритм последовательно обходит все компоненты связности, рекурсивно посещая непосещенные смежные вершины. Результатом работы является порядок обхода вершин графа.

3. 7LR_2_star.cpp (2 задание)

Программа реализует нерекурсивный обход графа в глубину с использованием стека. Граф представляется матрицей смежности, генерируемой случайным образом. Алгоритм начинает обход с заданной вершины, используя стек для хранения вершин. На каждом шаге из стека извлекается вершина, отмечается как посещенная, а ее непосещенные смежные вершины добавляются в стек в обратном порядке для сохранения последовательности обхода.

Результаты работы программы

Результаты работы программы показаны на рисунках 2-3.

I. LR_11_12.cpp (1 задание 1-2)

```

Введите количество вершин графа (от 1 до 20): 20
Матрица смежности:
  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
1  0  1  1  0  1  1  1  1  0  0  0  0  0  1  0  1  1  1  0  1
2  1  0  0  0  0  1  1  1  1  0  1  0  0  0  1  1  1  0  1  1
3  1  0  0  1  1  0  0  1  0  1  0  0  0  0  1  0  1  0  1  0
4  0  0  1  0  0  0  0  0  1  0  1  0  1  0  0  0  0  0  0  1
5  1  0  1  0  0  1  1  0  1  0  1  0  1  1  0  0  1  0  1  1
6  1  1  0  0  1  0  1  0  0  1  1  0  1  1  1  1  1  0  1  0
7  1  1  0  0  1  1  0  0  1  0  0  0  0  1  0  0  0  1  0  1
8  1  1  1  0  0  0  0  0  0  1  0  1  0  0  1  0  0  0  0  0
9  0  1  0  1  1  0  1  0  0  0  0  1  1  1  0  1  1  0  0  1
10 0  0  1  0  0  1  0  1  0  0  0  0  1  1  0  0  0  0  0  1
11 0  1  0  1  1  1  0  0  0  0  0  0  0  1  0  1  0  0  0  1
12 0  0  0  0  0  0  0  1  1  0  0  0  0  1  0  0  1  0  1  1
13 0  0  0  1  1  1  0  0  1  1  0  0  0  1  1  1  0  1  0  0
14 1  0  0  0  1  1  1  0  1  1  1  1  1  1  0  0  1  1  0  1
15 0  1  1  0  0  1  0  1  0  0  0  0  1  0  0  0  1  0  0  0
16 1  1  0  0  0  1  0  0  1  0  1  0  1  1  0  0  1  0  1  1
17 1  1  1  0  1  1  0  0  1  0  0  1  0  1  1  1  1  0  1  1
18 1  0  0  0  0  0  1  0  0  0  0  0  1  0  0  0  1  0  1  1
19 0  1  1  0  1  1  0  0  0  0  0  1  0  0  0  1  1  1  0  0
20 1  1  0  1  1  0  1  0  1  1  1  1  0  1  0  1  1  1  0  0
Порядок обхода в глубину: 1 2 6 5 3 4 9 7 14 10 8 12 17 15 13 16 11 20 18 19

```

Рисунок 1 – Корректная работа программы

```

Введите количество вершин графа (от 1 до 20): -5
Ошибка: количество вершин должно быть от 1 до 20

```

Рисунок 2 – Попытка ввести отрицательное число

II. 7LR_13.cpp (2 задание 1)

```

Введите количество вершин графа (от 1 до 20): 20
Списки смежности:
Вершина 1: 16 15 13 9 7 4 2
Вершина 2: 20 18 17 16 15 14 13 7 6 1
Вершина 3: 16 15 12 11 10 9 8 7
Вершина 4: 19 17 16 15 13 11 10 7 6 5 1
Вершина 5: 19 15 14 11 10 9 8 6 4
Вершина 6: 20 17 15 12 11 8 5 4 2
Вершина 7: 17 12 11 10 9 8 4 3 2 1
Вершина 8: 20 19 18 17 15 11 10 7 6 5 3
Вершина 9: 20 19 17 15 14 7 5 3 1
Вершина 10: 20 19 18 17 16 13 12 11 8 7 5 4 3
Вершина 11: 20 19 17 15 13 10 8 7 6 5 4 3
Вершина 12: 19 17 16 10 7 6 3
Вершина 13: 20 15 11 10 4 2 1
Вершина 14: 20 19 18 9 5 2
Вершина 15: 18 17 16 13 11 9 8 6 5 4 3 2 1
Вершина 16: 19 15 12 10 4 3 2 1
Вершина 17: 20 19 18 15 12 11 10 9 8 7 6 4 2
Вершина 18: 19 17 15 14 10 8 2
Вершина 19: 18 17 16 14 12 11 10 9 8 5 4
Вершина 20: 17 14 13 11 10 9 8 6 2
Порядок обхода в глубину: 1 16 19 18 17 20 14 9 15 13 11 10 12 7 8 6 5 4 2 3

```

Рисунок 3 – Корректная работа программы

```
Введите количество вершин графа (от 1 до 20): -7
Ошибка: количество вершин должно быть от 1 до 20
```

Рисунок 4 – Попытка ввести отрицательное число

III. 7LR_2_star.cpp (2 задание)

```
Матрица смежности:
  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
1  0  0  0  1  1  0  1  0  0  0  0  1  1  1  1  0  1  1  0  1
2  0  0  0  1  0  1  1  0  1  0  0  0  1  0  0  1  1  1  0  1  1
3  0  1  0  0  0  0  1  0  1  1  1  1  1  1  1  1  0  0  0  0  0
4  1  0  0  0  0  1  0  1  0  1  1  0  0  1  1  1  1  1  0  0  0
5  1  1  0  1  0  1  1  1  0  0  0  1  1  1  1  1  1  0  0  0  0
6  0  1  1  0  1  0  1  0  1  1  0  1  0  0  1  1  1  1  0  0  1
7  1  0  0  1  1  1  0  0  0  1  0  0  1  0  1  1  1  1  0  1  1
8  0  1  1  0  1  0  0  0  0  0  0  1  1  1  0  0  0  1  0  1
9  0  0  1  1  0  1  0  0  0  1  0  1  0  1  1  0  1  1  0  0
10 0  0  1  1  0  1  1  0  1  0  0  1  0  0  0  1  0  1  0  0
11 0  0  1  0  0  0  0  0  0  0  0  1  1  1  1  1  1  1  0  0
12 1  1  1  0  1  1  0  1  1  1  1  0  1  0  1  0  0  0  0  1
13 1  1  0  1  1  0  1  1  0  0  1  1  0  1  0  1  1  0  1  0
14 1  0  1  1  1  0  0  1  1  0  1  0  1  0  1  0  1  0  0  0
15 1  1  1  1  1  1  1  0  1  0  1  1  0  1  0  1  1  1  1  1
16 0  1  0  1  1  1  1  0  0  1  1  0  1  0  1  0  0  0  1  0
17 1  1  0  1  0  1  1  0  1  0  1  0  1  1  1  0  0  1  0  0
18 1  0  0  0  0  0  0  1  1  1  1  0  0  0  1  0  1  0  1  0
19 0  1  0  0  0  0  1  0  0  0  0  0  1  0  1  1  0  1  0  0
20 1  1  0  0  0  1  1  1  0  0  0  1  0  0  1  0  0  0  0  0
Порядок обхода в глубину (нерекурсивный): 1 4 9 3 2 19 8 11 6 10 16 5 7 12 13 14 15 17 18 20
```

Рисунок 5 – Корректная работа программы

```
Введите количество вершин графа (от 1 до 20): -10
Ошибка: количество вершин должно быть от 1 до 20
```

Рисунок 6 – Попытка ввести отрицательное число

Выводы

В ходе выполнения лабораторной работы были успешно реализованы и протестированы различные подходы к обходу графов в глубину. Разработанная программа включает три основные реализации: рекурсивный обход для матрицы смежности, рекурсивный обход для списков смежности и нерекурсивный обход с использованием стека. Особенностью реализации является генерация случайных графов с возможностью наличия петель и обеспечение корректного обхода всех компонент связности. Интерфейс программы предоставляет удобный выбор типа представления графа и метода обхода с демонстрацией результатов в наглядной форме. В процессе работы подтверждена эквивалентность результатов обхода для различных представлений графа, что свидетельствует о корректности реализованных алгоритмов и их устойчивой работе при различных структурах входных данных.